Studio Nx

Backend(FastAPI/Python) 27 — create_refresh_token()

2026-10-07

Backend(FastAPI/Python) 27 — `create_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인 — Refresh Token 발급 · PRD 01 FR-105 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	26 `create_access_token()` · 28 `decode_refresh_token()`

1. 하는 일

회원 번호로 **3일짜리 Refresh Token(JWT)** 을 만든다. 26 과 같은 모양이고 종류와 수명만 다르다.

	Access Token (26)	Refresh Token (이 함수)
`type`	`"access"`	`"refresh"`
수명	30분	**3일** (`jwt_refresh_days`)
브라우저 어디에	JS 메모리 (`api.js` 변수)	**HttpOnly 쿠키** `snx_refresh` — JS 가 읽을 수 없음
서버로 가는 곳	모든 API 의 `Authorization: Bearer …`	`/api/v1/auth/…` 요청에만 쿠키로 자동으로 실림
쓰임	API 요청	새 Access Token 받기 (33 `/auth/refresh`)

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/login` (32)	로그인 성공 → 이 토큰을 쿠키에 담아 응답 (31 `set_refresh_cookie`)

두 토큰이 함께 움직이는 모습

순서	일어나는 일
①	로그인 → Access(30분)는 응답 본문, Refresh(3일)는 쿠키
②	30분 동안 API 요청은 Access 로
③	Access 만료 → 서버 401 → 브라우저가 `POST /auth/refresh` (쿠키가 자동으로 실림)
④	서버가 Refresh 를 검사(28) → 새 Access(26)
⑤	3일이 지나면 Refresh 도 만료 → 다시 로그인

자동 로그인 체크와의 관계: 토큰의 수명은 **늘 3일**이다. 체크 여부는 **쿠키**의 수명만 바꾼다 (31).

로그인 모달의 "Keep me signed in"	쿠키	결과
체크	3일 쿠키	브라우저를 닫았다 열어도 3일 동안 로그인 유지
체크 안 함	세션 쿠키 (수명 없음)	브라우저를 닫으면 쿠키가 사라짐 → 다시 로그인

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다 (26 과 같음, 이번에 추가한 것 없음). 이 함수는 **`timedelta`** · **`get_settings`** 를 쓰고, 만드는 일은 26 의 `_create_token()` 에 맡긴다.

함수

```
def create_refresh_token(user_id: int) -> str:
    days = get_settings().jwt_refresh_days
    return _create_token(user_id, "refresh", timedelta(days=days))
```

문법 풀이

import 문법	뜻
`from datetime import timedelta`	기간을 나타내는 타입 (26 에서 이미 가져옴)

식	뜻
`get_settings().jwt_refresh_days`	`.env` 설정. 기본 3 (`core/config.py`)
`timedelta(days=3)`	3일 = 259,200초
`_create_token(user_id, "refresh", …)`	26 과 같은 함수에 종류 · 수명만 다르게 넘김

4. 흐름

1. 설정에서 수명(3일)을 읽는다.
2. 26 의 `_create_token()` 으로 `type: "refresh"` 토큰을 만든다.

5. 왜 이렇게

선택	이유
Access 와 나눔	자주 오가는 Access 는 짧게(30분), 오래 가는 Refresh 는 JS 가 못 읽는 쿠키에 숨긴다. 스크립트 공격(XSS)이 Access 를 훔쳐도 30분 뒤면 못 쓴다
`type: "refresh"`	05 가 Access 자리에서 이 토큰을 거절하고, 28 은 반대로 Access 를 거절한다. 두 토큰을 서로 바꿔 쓸 수 없다
토큰 수명은 체크와 상관없이 3일	쿠키가 브라우저에서 사라지면 토큰도 쓸 일이 없다. 토큰을 두 가지로 만들 필요가 없다

선택	이유
같은 비밀값 · 알고리즘	26 과 같은 `_create_token` 을 써서 서명 방식이 하나로 유지된다

6. 시험 (`tests/test_security.py` 에 3개 추가 · 모두 17개)

시험	확인한 것
내용	`sub` = `"7"`, `type` = `"refresh"`, `exp - iat` = 259,200초 (3일)
Access 자리에 쓰기	05 `decode_access_token` 이 401 — 3일짜리 토큰으로 API 를 부를 수 없음
두 토큰은 다름	같은 회원의 Access 와 Refresh 가 다름

전체 시험 194개 통과, `ruff` 검사 통과.